

CCDV-F Topic Summaries

One paragraph per topic area — use for a fast final review pass.

Agent architecture

The gating question is whether the steps of a task can be mapped out in advance: if yes, build a workflow (execution follows a predefined code path you control); if no, build an agent (the LLM dynamically directs its own process and tool use). When unsure, start with an agent and extract workflow patterns as they emerge from real usage rather than guessing a structure up front. Anthropic names five workflow patterns — prompt chaining (sequential, each step processing the last), routing (classify then dispatch to a specialized handler), parallelization (sectioning for speed or voting for confidence), orchestrator-workers (a central LLM dynamically decomposes unpredictable subtasks and delegates them — the bridge pattern between workflow and agent), and evaluator-optimizer (one LLM generates, another critiques and drives refinement). Multi-agent systems typically organize around a supervisor pattern: a central coordinator delegates to workers and reviews their output without executing the work itself, sometimes extended into a multi-level hierarchical variant. Subagents improve execution along three independent axes regardless of pattern: context isolation (only the final summary returns to the parent), parallelization (independent subtasks finish in the slowest one's time, not the sum), and specialization (narrow system prompts and scoped tool access reduce both noise and blast radius). Orchestrator-workers has a concrete, testable cost: Anthropic's own reported multi-agent research system runs at roughly 15x the token cost of a single-agent chat, and that multiplier only pays off when the task genuinely decomposes into independent parallel parts — on tightly-coupled work like coding, subagents mostly wait on each other and the fan-out cost buys nothing. Human-in-the-loop placement follows the same worst-case-cost logic as permission-mode selection: gate before any destructive/irreversible tool call, after a planning step whose direction matters, and on any unexpected tool output — the design question is always "what's the worst outcome if this runs unchecked," asked at scoping time, not after the first incident. Agent memory has three scopes — in-context (one continuous session), external storage (the same task continuing across many separate shorter sessions), and stateless (fully independent jobs) — and choosing the wrong one is a common dev/production mismatch, since a session shape that works as one long dev session can silently accumulate and overflow across many short production sessions.

Agent construction and deployment

The Claude Agent SDK embeds Claude Code's own execution loop — receive prompt, evaluate/respond, execute tools, repeat until no tool calls remain, return result — programmable via `ClaudeAgentOptions / Options` (tool allow/deny lists, `permission_mode`, `effort`, `max_turns` / `max_budget_usd` as runaway-loop guards). Subagents are defined via `AgentDefinition` (description, prompt, tools, model, effort, max_turns) either programmatically or as `.claude/agents/` markdown files, and their budget/spend rolls up into the parent's cap. Three wiring paths trade off how much runtime infrastructure you own: a raw Messages API loop (full control, full responsibility), the Agent SDK (loop/context/tool scaffolding provided, running in your own process), and Managed Agents (Anthropic runs the loop and sandbox server-side, you define the agent once and stream events) — with self-hosted sandboxes as a middle ground that keeps orchestration on Anthropic's side while tool execution stays in your infrastructure. The governing constraint that overrides operational preference: Managed Agents' server-side stateful sessions are not currently eligible for Zero Data Retention or a HIPAA BAA, so a PHI or ZDR requirement rules that path out regardless of how well it otherwise fits a long-running, sandbox-avoidant workload. Hooks — `PreToolUse`, `PostToolUse`, `UserPromptSubmit`, `Stop`, `SubagentStart` / `SubagentStop`, `PreCompact` — run in your own process (zero context-window cost) and provide deterministic, code-level

enforcement a prompt-level instruction can't guarantee; a `PreToolUse` deny short-circuits the loop entirely, including under `bypassPermissions`. Third-party frameworks (LangGraph for explicit graph-based state control, PydanticAI for Python type safety without heavy orchestration, Strands for AWS/Bedrock-native model-driven agents) are independent, cross-provider alternatives to the SDK's provider-native primitive, reached for when you need a pattern the SDK doesn't provide natively, not because the SDK's core loop is less capable.

Claude API mechanics and application design

The Messages API is stateless per request — your application owns and resends the full conversation state, including tool-use/tool-result pairs matched by `tool_use_id`. Vision input comes via `base64` (simplest, but re-sent in full every turn), `url`, or Files-API `file_id` (best for multi-image conversations, avoids compounding payload size). Extended thinking and adaptive thinking are distinct mechanisms that don't overlap on the same model — Fable 5/Opus 5/Sonnet 5 use adaptive thinking (depth tuned via `effort`), Haiku 4.5 uses extended thinking (`thinking.type: "enabled"`) — a frequently-tested distinction. Claude reaches users through six deployment surfaces — the first-party API, Claude Platform on AWS, Claude in Amazon Bedrock, legacy Bedrock (InvokeModel/Converse), Google Vertex AI, and third-party platforms like Microsoft Foundry — chosen primarily by the customer's existing cloud and compliance posture rather than technical merit, with real API-shape differences to account for when porting code (Google Cloud specifies the model in the endpoint URL and requires an `anthropic_version` field; Bedrock uses its own model-ID scheme with global-vs-regional endpoint residency control; Foundry has two hosting forms — Azure-hosted vs. Anthropic-hosted — with different EU-residency properties per model, not per platform). Platform choice is defensible only when measured on three dimensions: latency from the customer's actual region, compliance against their existing certification (usually pass/fail for regulated customers, not a tradeoff), and total cost per call including egress and integration. Requirements themselves split into functional (checkable behavior, derived from a business problem, not the problem statement itself) and infrastructure (latency/scale/residency/identity), captured before design and gated through a seven-phase lifecycle (requirements → design → build → test → deploy → operate → iterate) where a "gate" — like not entering build until the platform satisfies residency — is what keeps a regulated engagement reviewable. Version pinning matters the same way across all six surfaces: an alias resolves to a moving target that can differ by platform, while a full pinned model ID is fixed until deliberately changed, with promotion gated on the eval suite against a retained baseline and a prior version kept available for rollback. The realtime-vs-batch decision (sync/streaming/async client vs. the Message Batches API — up to 100,000 requests or 256MB per call, results returned in arbitrary order matched via `custom_id`, up to 24h turnaround at lower per-token cost) recurs across the exam as the canonical "which infrastructure pattern" test case; chunking a loop over the synchronous endpoint is not batching and hits the same rate limits regardless of chunk size. Application design spans how Claude receives persistent instructions differently per interface (CLAUDE.md hierarchy in Claude Code vs. an explicitly-managed `system` parameter over the raw API, with zero implicit persistence in the latter), content-boundary decisions (what's instruction-bearing vs. inert data — the same discipline underlying indirect-injection defense), schema design as an API contract deserving the same change-management rigor as a public API, session hygiene (deciding session lifetime, when to fork/summarize, cleanup of stale sessions), and plugin management (`.claude-plugin/plugin.json` manifests, marketplace-driven dependency auto-install). Configuration management ties it together: CLAUDE.md, settings.json, model version pinning (every model ID is a pinned snapshot — pinning protects production from unannounced behavior shifts), and prompt versioning all deserve the same version-control-and-eval-validated discipline as code, because none of them are compiled or type-checked.

Claude Code operation

Claude Code's five core components are Rules (CLAUDE.md), Skills (`.claude/skills/<name>/SKILL.md`), Commands (`.claude/commands/<name>.md`), Agents (`.claude/agents/<name>.md`), and Agent Memory — commands and skills are two authoring surfaces for the same underlying slash-invocation mechanism. `settings.json` resolves through a fixed precedence — managed (org IT, cannot override) > CLI flags > local (personal, gitignored) > project (checked in, team-shared) > user (personal, global) — with one documented exception: `permissions` rules accumulate across every applicable scope rather than the highest scope simply overriding lower ones. Session modes split into headless/print mode (`-p` , for CI/scripting, with `--bare` skipping auto-discovery of hooks/skills/plugins/MCP/CLAUDE.md for deterministic, faster CI runs at the cost of losing OAuth/keychain access), streaming mode (`--output-format stream-json` plus `--include-partial-messages` for token-level deltas), and auto-mode (a model classifier approves/denies permission prompts automatically, distinct from `bypassPermissions` , which skips evaluation entirely). Six permission modes trade oversight for speed by risk level — `default` (prompts nearly everything) through `plan` , `acceptEdits` , `auto` (classifier-reviewed), `dontAsk` (allow-list only), to `bypassPermissions` (isolated environments only, uniquely skips the protected-path guard the other five keep) — and mode choice is a risk decision, not a convenience one. Rules instruction files (`.claude/rules/` , scoped via a `paths` glob in frontmatter) add a narrower layer beneath CLAUDE.md for path-specific guidance that would be noise everywhere else, while CLAUDE.md's own failure mode is dilution — a correct rule buried among hundreds of accumulated lines gets a smaller effective share of attention even though it's technically present.

Debugging and error handling

The Claude API's HTTP status codes map predictably to error types and retry behavior: 400/401/402/403/413 are non-retryable (fix the request, credential, billing, permission, or payload size before trying again), while 429 (`rate_limit_error`) and 529 (`overloaded_error`) are retryable with exponential backoff, honoring any `retry-after` header. The Agent SDK layers its own loop-level failure subtypes on `ResultMessage` — `error_max_turns` , `error_max_budget_usd` , `error_during_execution` , `error_max_structured_output_retries` — which describe why the agent loop stopped, a distinct concept from why a single HTTP call failed. The central debugging discipline is isolating whether a failure originates in the integration layer (malformed request, wrong tool schema, mismatched `tool_use_id` pairing, a hook silently denying a call — diagnosed via the raw request/response and the harness's event stream) or in the model's output itself (wrong, incomplete, or oddly-shaped response — diagnosed via the prompt-engineering failure-signature table, not by inspecting request-construction code). A `stop_reason` of `refusal` or `max_tokens` is specifically a signal to examine what was asked for, not the surrounding integration code.

Model fundamentals, selection, and cost

Tokens (not characters or words) are the unit of everything Claude processes and every cost/budget calculation, with the chars-per-token ratio varying by tokenizer across model generations — never hardcode a conversion constant. The context window is a fixed, shared budget across system prompt, history, injected documents, tool results, and output, with two distinct failure modes: an oversized input is rejected with a validation error before generation starts, while a request that fits but hits the ceiling mid-generation stops and returns partial output (`model_context_window_exceeded`). Sampling explains non-determinism: the model samples from a probability distribution rather than picking one fixed next token, so identical prompts can yield different (equally valid) wording — the newest top-tier models don't even accept `temperature / top_p / top_k` (400 error), steering behavior through prompting instead, and even where accepted, `temperature: 0` improves repeatability without guaranteeing identical output. This is why evals (property assertions, model-graded judges) replace exact-text test assertions for anything Claude generates. The current model lineup trades cost/latency/capability across four tiers — Fable 5 (most capable, always-on

adaptive thinking, 1M context), Opus 5 (complex agentic/enterprise work), Sonnet 5 (default speed/intelligence balance), and Haiku 4.5 (fastest/cheapest, notably using extended thinking rather than adaptive thinking, 200K context) — with the practical selection workflow being start at Sonnet, move up only when an eval shows a quality gap, move down to Haiku only when an eval shows the drop is acceptable. Reasoning mode is an orthogonal per-request decision from model choice: adaptive thinking is tuned via `effort` (low/medium/high/xhigh/max), and the older `budget_tokens` control is deprecated, returning a 400 error on the newest generations. Cost management leans heavily on prompt caching — automatic (one top-level `cache_control`, breakpoint slides forward automatically) or explicit (up to 4 breakpoints per request, 20-block lookback, minimum token threshold), at a 5-minute or pricier 1-hour TTL depending on traffic pattern.

Prompt and context engineering

Context engineering is the superset of prompt engineering: prompting is about what you say, context engineering is about curating the entire token budget (prompt, history, tool definitions, tool outputs) as a finite, shared resource. Two distinct techniques manage long-session bloat: pruning/clearing (cheap, lossless — for re-fetchable tool output whose immediate value has passed) and compaction (an LLM-driven summarization pass for dialogue/reasoning that can't be cheaply re-fetched, triggered automatically as the window approaches its limit, with a `PreCompact` hook available to archive the full transcript first and persistent rules belonging in `CLAUDE.md` since it's re-injected every request rather than lost to summarization). Subagents provide the cleanest architectural tool for context isolation on long or exploratory tasks. The core prompt-engineering discipline is diagnosis over elaboration: a prompt that breaks in production needs the *missing structural technique* identified and added, not more wording — wrong output shape signals a missing output constraint, content drift signals an underspecified system prompt, invented structure signals missing few-shot examples, and edge-case failure signals a missing constraint for that specific variant; XML tags delimit few-shot examples from live instructions. Output handling moves this from best-effort to guaranteed via structured outputs: JSON outputs (`output_config.format`, `json_schema`) constrain the final response, strict tool use (`strict: true`) constrains tool arguments, both via constrained decoding that makes an invalid token literally ungeneratable — though refusals and truncation can still produce a non-parsing response despite the schema, first-request grammar-compilation adds latency (cached 24h), and the feature is incompatible with message prefiling.

Security, safety, and identity

Two threat models drive prompt-injection defense: direct injection (the application's own user is the adversary) mitigated with harmlessness screens, input validation, hardened system prompts, and repeat-offender throttling; and indirect injection (a trusted user, but Claude processes adversarial third-party content embedded in a document, email, or tool result) mitigated structurally — isolate untrusted content in `tool_result` blocks only, label its nature/source, state the untrusted-content policy explicitly in the system prompt, JSON-encode it to prevent delimiter breakout, never place your own instructions inside a tool result (send them in the following user turn instead), screen tool outputs with a lightweight classifier before Claude acts on them, and bound the blast radius with least privilege regardless of whether the other layers hold. No single guardrail is sufficient alone — treat the whole stack as defense-in-depth, never a substitute for input validation, least-privilege tool scopes, or human approval on destructive actions. Hooks are the deterministic enforcement layer for all of this: a `PreToolUse` hook can deny a dangerous command outright, enforcing even under `bypassPermissions`, but only if it exits with code 2 — exit code 1 merely warns without blocking, a frequent source of "the hook didn't actually stop it" bugs, and hook scripts need to stay fast since they run synchronously in the loop. Identity and key management closes the loop: API keys are shown once at creation and belong in a secrets manager/KMS immediately, rotated periodically and revoked on suspected leak, with short-lived federated credentials (e.g. Workload Identity Federation exchanging a platform OIDC token for a short-lived Anthropic token) preferred over long-lived static keys wherever the workload already

has a platform identity to federate from. In multi-component applications the same isolation discipline applies at every inter-component seam, not just the outermost boundary — a component that passes its own tests in isolation has no seam-level controls, and the application's overall containment is bounded by its single most-privileged component. Regulated data constraints (attorney-client privilege, HIPAA/PHI, GDPR/EU residency, FedRAMP, internal residency policy) each decide endpoint, credentials, and log destination before any prompt or tool is designed — concretely: the direct API has no EU residency (route through Bedrock/Vertex with region pinned), a HIPAA BAA excludes Console/beta/consumer plans, and only three specific routes (C4G, Bedrock GovCloud, Vertex Assured Workloads — not general AWS Marketplace) carry FedRAMP authorization.

Tools and MCP

Tool use lets Claude call functions you define (or built-in ones); with default `tool_choice: "auto"`, Claude decides per-turn whether a request maps to a tool's *description* closely enough to call it, versus responding directly for stable knowledge or conversation — making tool description quality a design task, not documentation. Every tool description needs two parts — when to use it and when not to — since two overlapping "use this to find information"-style descriptions give Claude no signal to route on; the fix is always an added exclusion sentence, or merging the tools with a `type` parameter if they can't be cleanly separated. A tool-use conversation is built from typed, API-enforced blocks (`text`, `tool_use`, `tool_result`, `thinking`) with a strict pairing rule — every `tool_use` needs a matching `tool_result` in the immediately following turn, and a `thinking` block's signature breaks (rejecting the request) if it's edited rather than passed back unchanged. Dispatch splits into client-side (your app executes the logic, returns `tool_result`) and server-side/harness dispatch (Claude Code or the Agent SDK loop executes built-in and MCP tools as part of the loop, you configure access rather than execution); parallel tool calls run read-only tools concurrently and state-mutating tools sequentially by default, with `readOnlyHint` opting a custom tool into concurrent execution. MCP (Model Context Protocol) solves reusability specifically: one independently-maintained server exposing resources (readable data), tools (callable functions), and prompts (templates) over stdio (local subprocess) or HTTP (networked/shared) transport can serve many different client applications, registered at local/project/user scope in Claude Code, with tool schemas deferred by default (loaded on demand rather than upfront) to avoid burning context on unused capability. The recurring exam framework is choosing among built-in tools (generic, already-available capability), custom tools (one app, one specific function, no reuse needed), Skills (procedural knowledge/judgment calls, no new live data access), and MCP servers (live/dynamic data or actions shared across multiple independently-maintained applications) — remembering that MCP connects Claude to data while Skills teach Claude what to do with that data, and that a production agent typically uses all three together rather than picking just one.